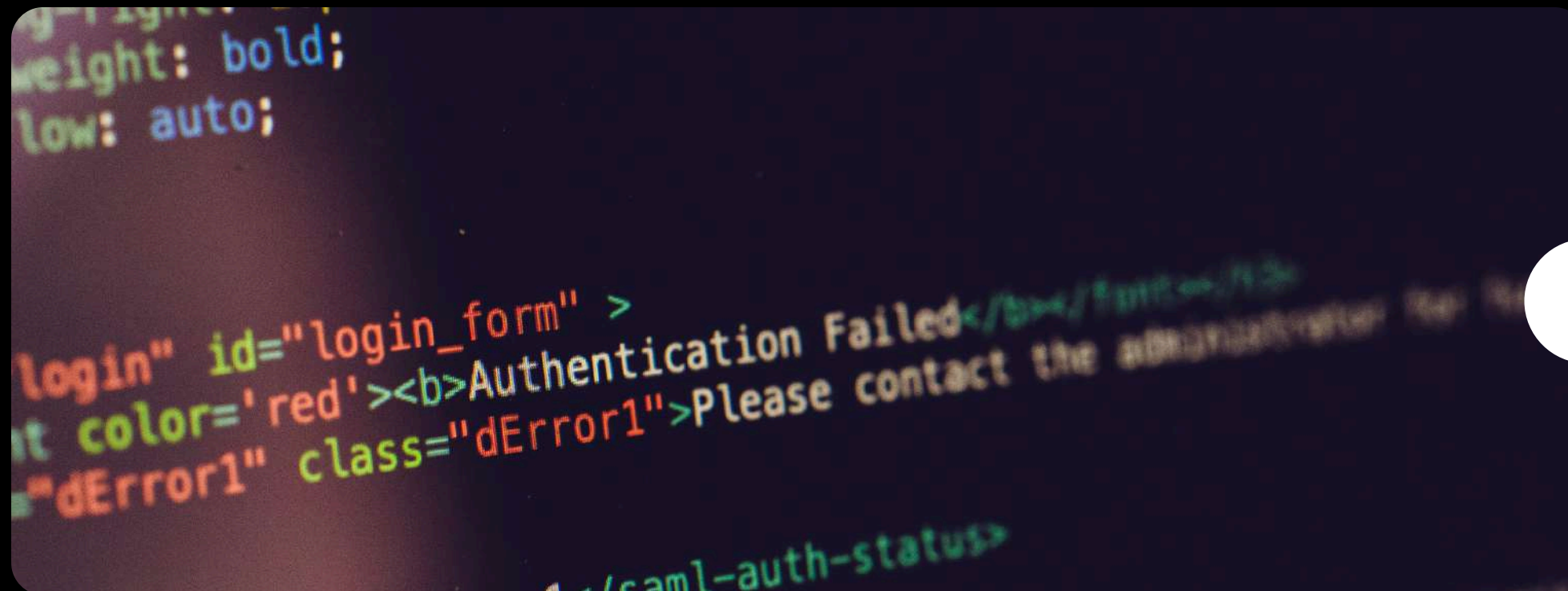
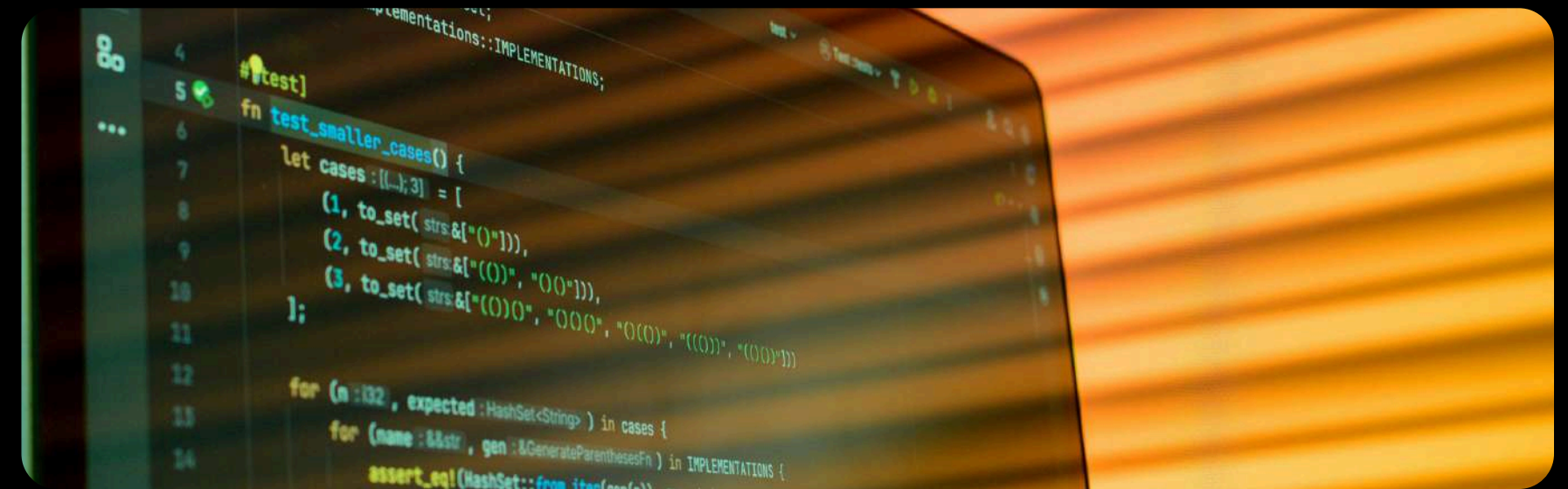


Проект JoyKeen

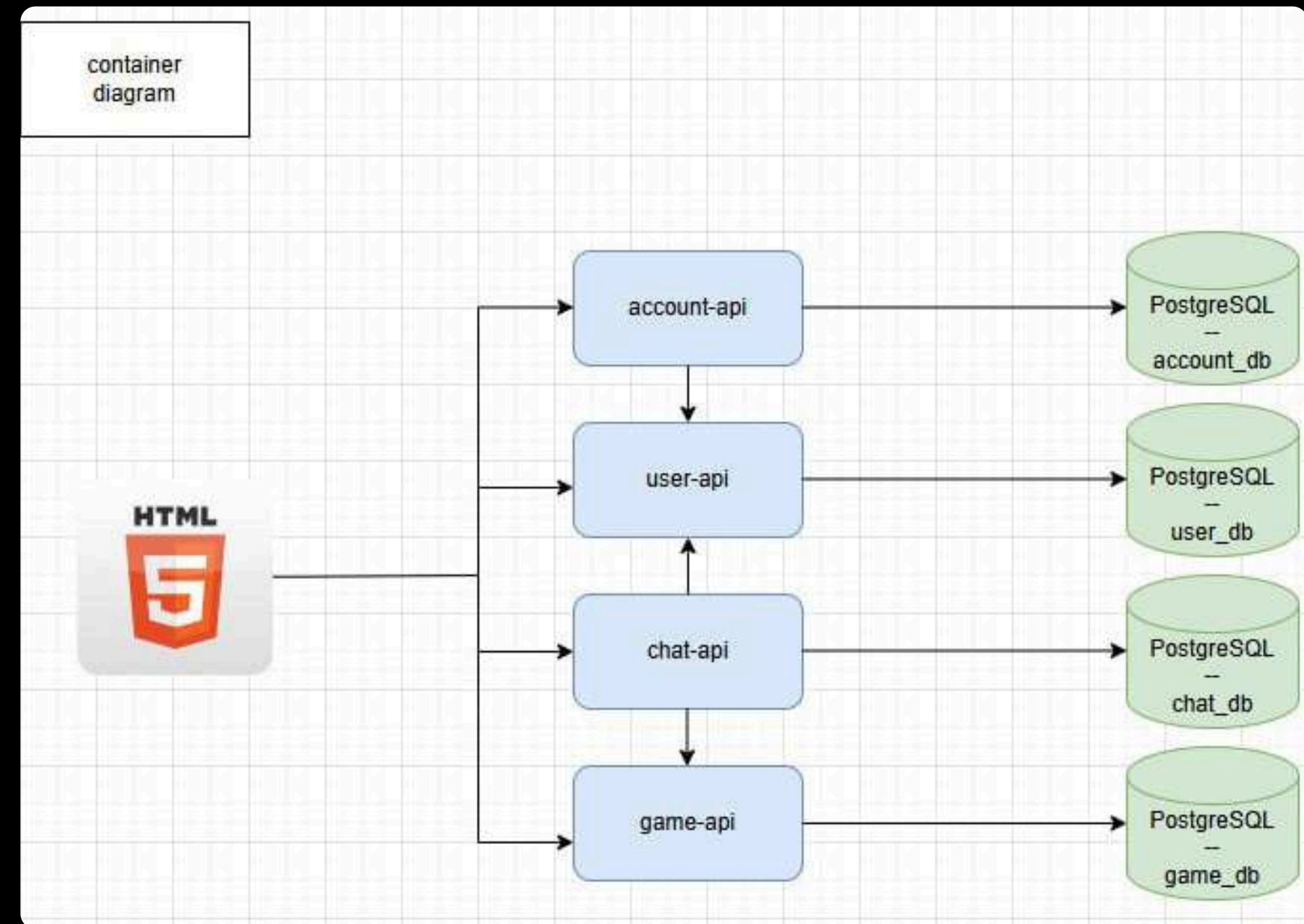


JoyKeen — це мікросервісна платформа для спілкування користувачів у реальному часі із можливістю запуску спільних ігрових модулів безпосередньо у вікні чату та збереженням статистики.

Кс-21 (Ліньков О.О, Мірошніченко О.А, Ковтун О.О, Шкарбан К.І.)

Ключові функціональні модулі системи:

1. Користувачі та контакти (users-api / account-api): Реєстрація, авторизація, керування профілями, пошук друзів за нікнеймами та робота зі списками контактів.
2. Чати та повідомлення (chat-api): Створення приватних і групових чатів, обмін текстовими повідомленнями в реальному часі, відображення статусів (онлайн/друк).
3. Ігрова екосистема (game-api): Перегляд доступних ігор, запуск ігрових сесій прямо в діалозі, запрошення учасників та синхронізація ігрового процесу.
4. Статистика та сповіщення: Фіксація результатів матчів (історія перемог/поразок) та система нотифікацій (про нові повідомлення, запити в друзі чи запрошення до гри).



Демонстрація розгорнутого функціоналу

Page 03

01

Перевірка через Swagger UI:

Після успішного деплою ми виконали тестування доданої фічі безпосередньо на живому середовищі. Запит до нового ендпоінту GET /info успішно направлено до кластера.

02

Коректна відповідь API:

Сервіс Account API опрацював запит, повернув статус 200 OK та очікуваний масив даних у форматі JSON (інформацію про ігри користувача), що повністю підтверджує працездатність оновленого коду.

03

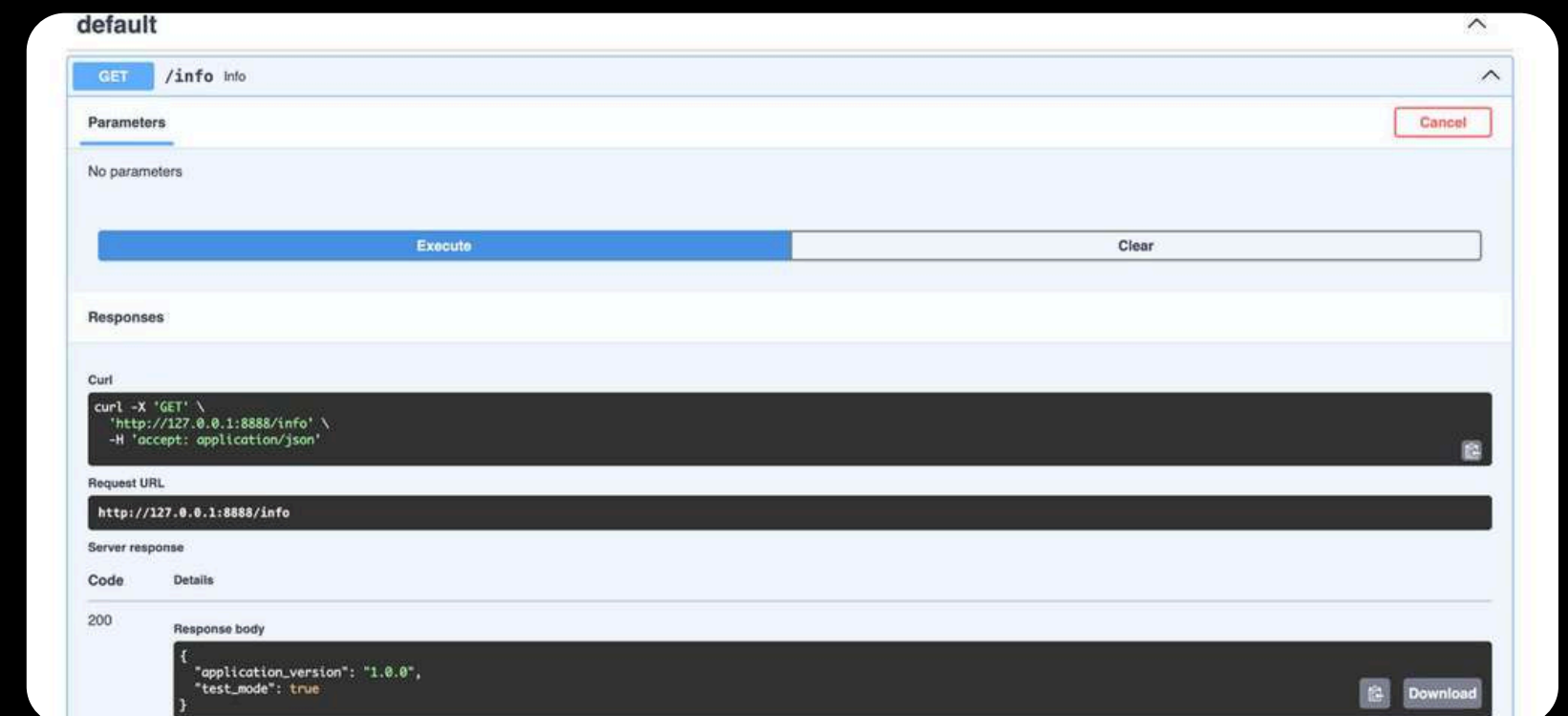
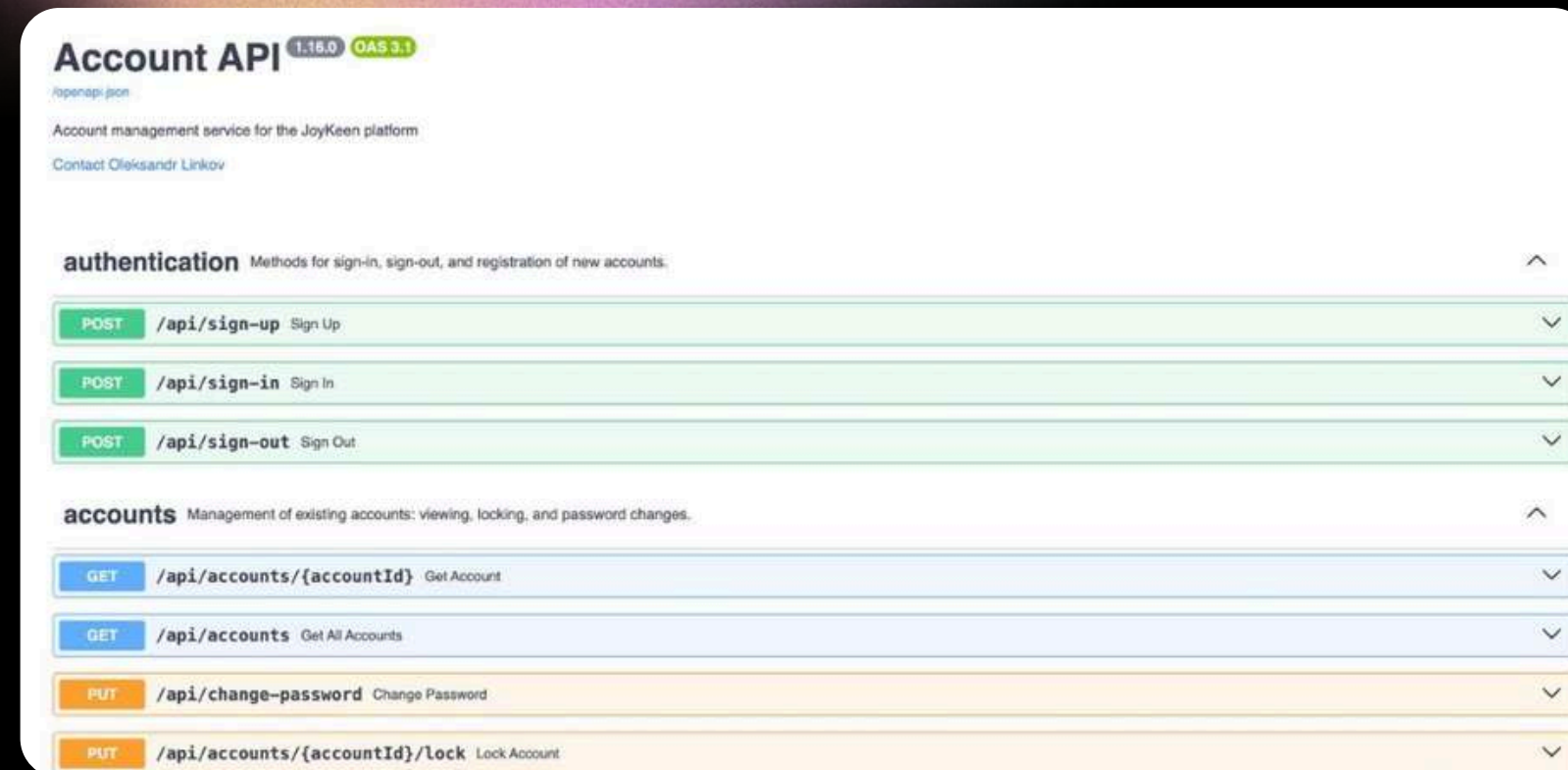
Валідація на рівні бази даних:

Для підтвердження цілісності системи ми звірили результати API з ізольованою базою даних PostgreSQL. У DBeaver чітко видно таблицю games, де зберігаються відповідні записи.

04

Успішне завершення циклу:

Дані, отримані через клієнтський запит, на 100% збігаються з фізичним станом бази даних.



Демонстрація розгорнутого функціоналу

Page 04

01

Перевірка через Swagger UI:

Після успішного деплою ми виконали тестування доданої фічі безпосередньо на живому середовищі. Запит до нового ендпоінту GET /games/user/{user_id} успішно направлено до кластера.

02

Коректна відповідь API:

Сервіс Game API опрацював запит, повернув статус 200 OK та очікуваний масив даних у форматі JSON (інформацію про ігри користувача), що повністю підтверджує працездатність оновленого коду.

03

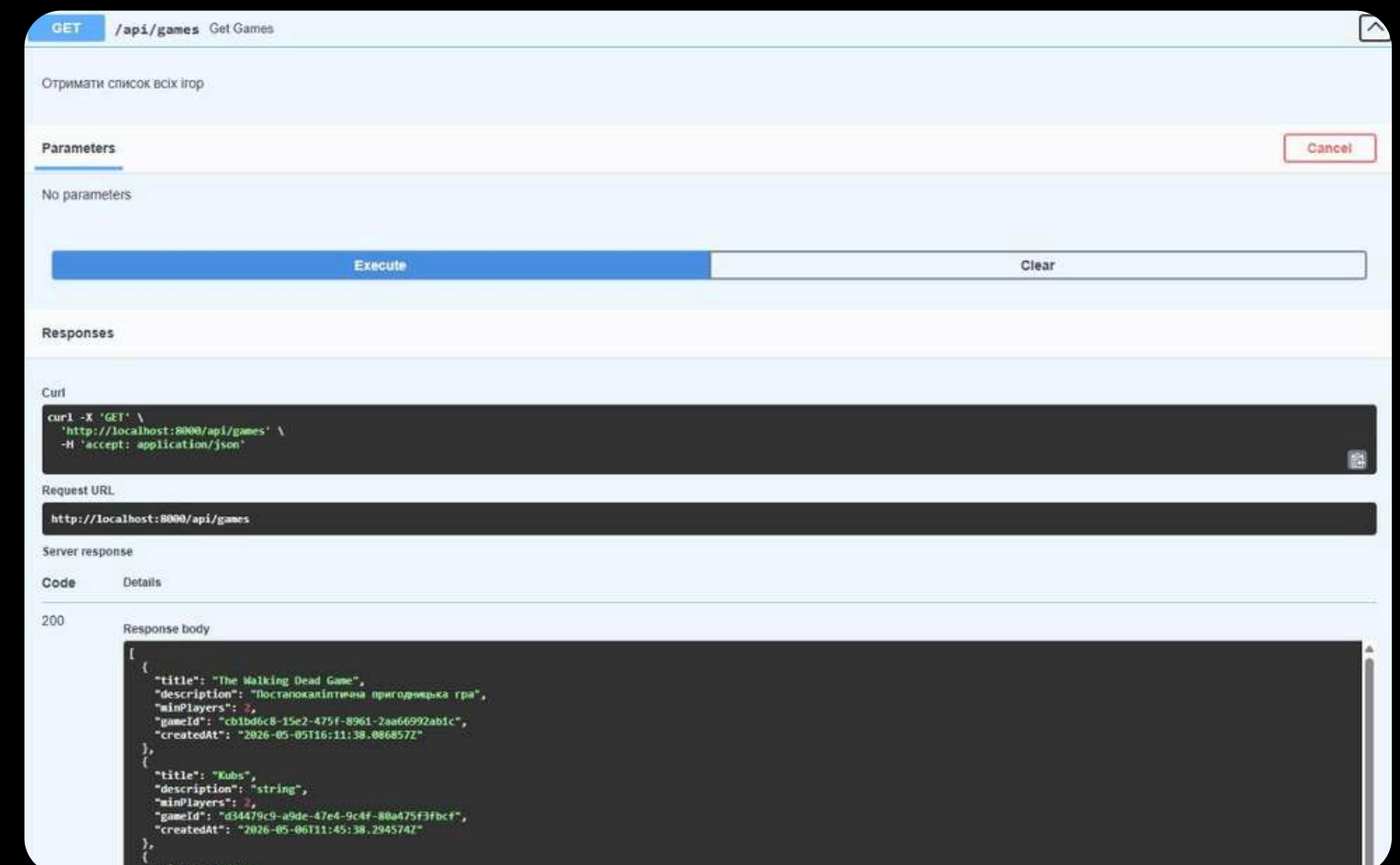
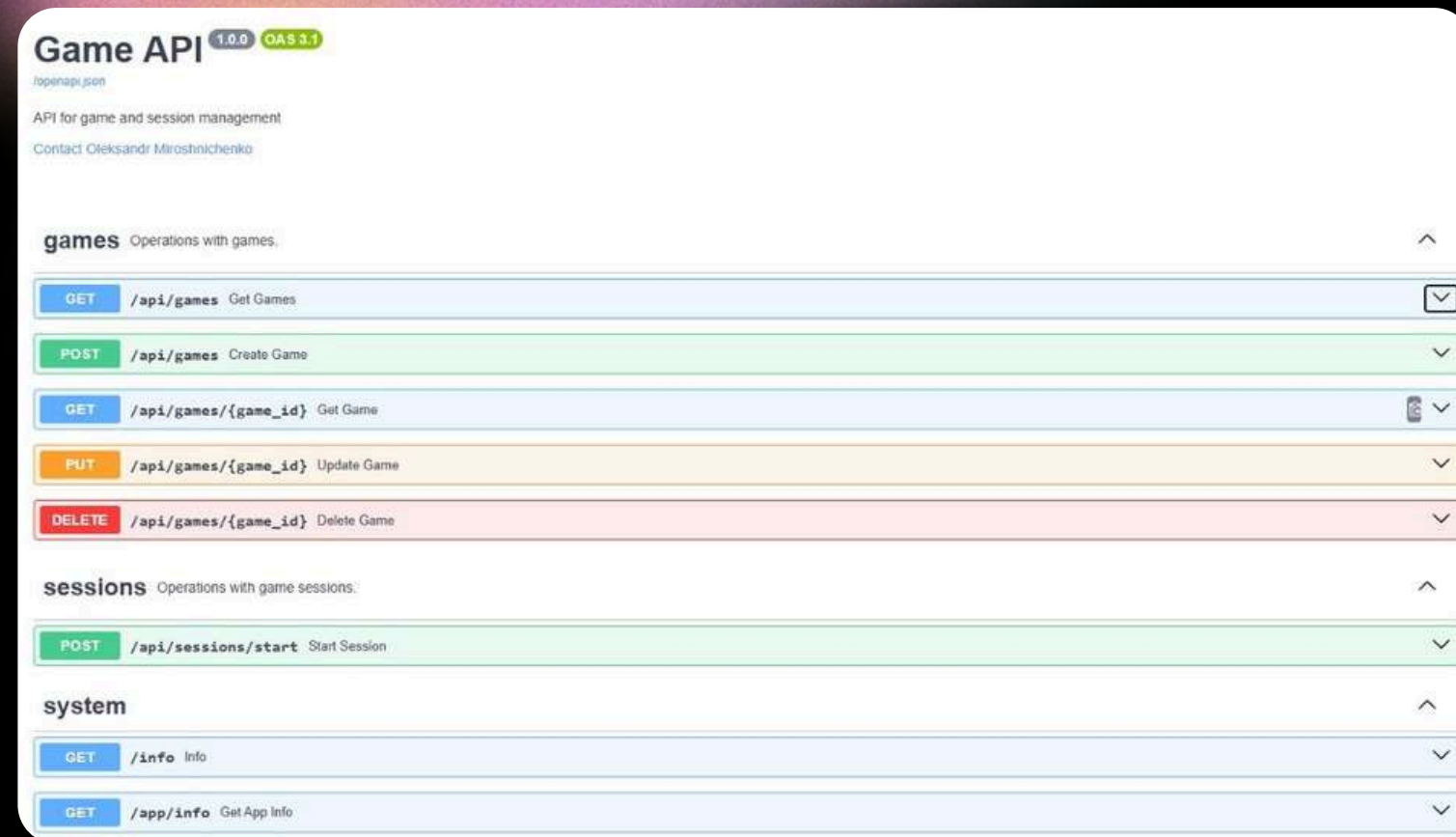
Валідація на рівні бази даних:

Для підтвердження цілісності системи ми звірили результати API з ізольованою базою даних PostgreSQL. У DBeaver чітко видно таблицю games, де зберігаються відповідні записи.

04

Успішне завершення циклу:

Дані, отримані через клієнтський запит, на 100% збігаються з фізичним станом бази даних.



Демонстрація розгорнутого функціоналу

01

Перевірка через Swagger UI:

Після успішного деплою ми виконали тестування доданої фічі безпосередньо на живому середовищі. Запит до нового ендпоінту GET /users успішно направлено до кластера.

02

Коректна відповідь API:

Сервіс Users API опрацював запит, повернув статус 200 OK та очікуваний масив даних у форматі JSON (інформацію про ігри користувача), що повністю підтверджує працездатність оновленого коду.

03

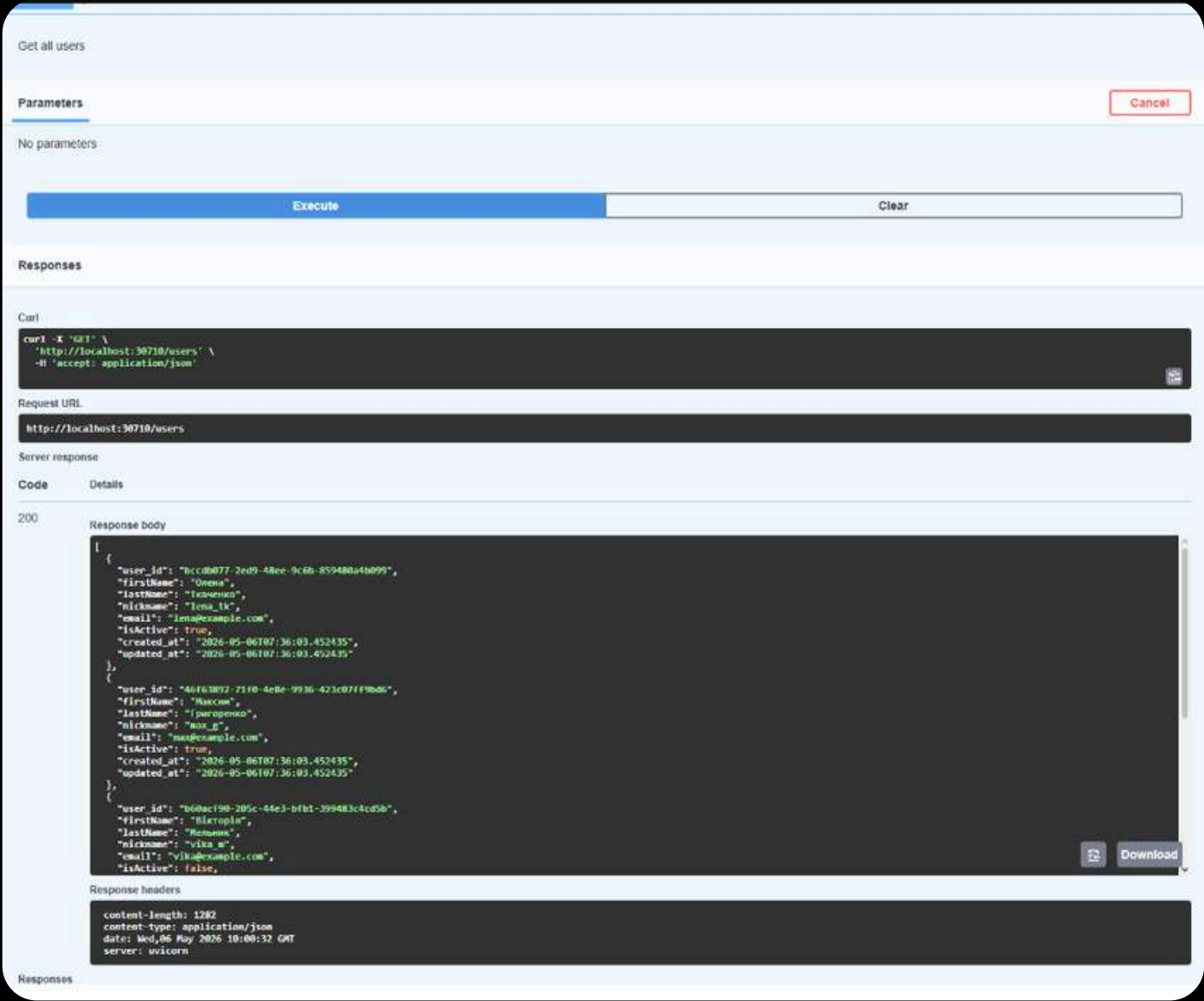
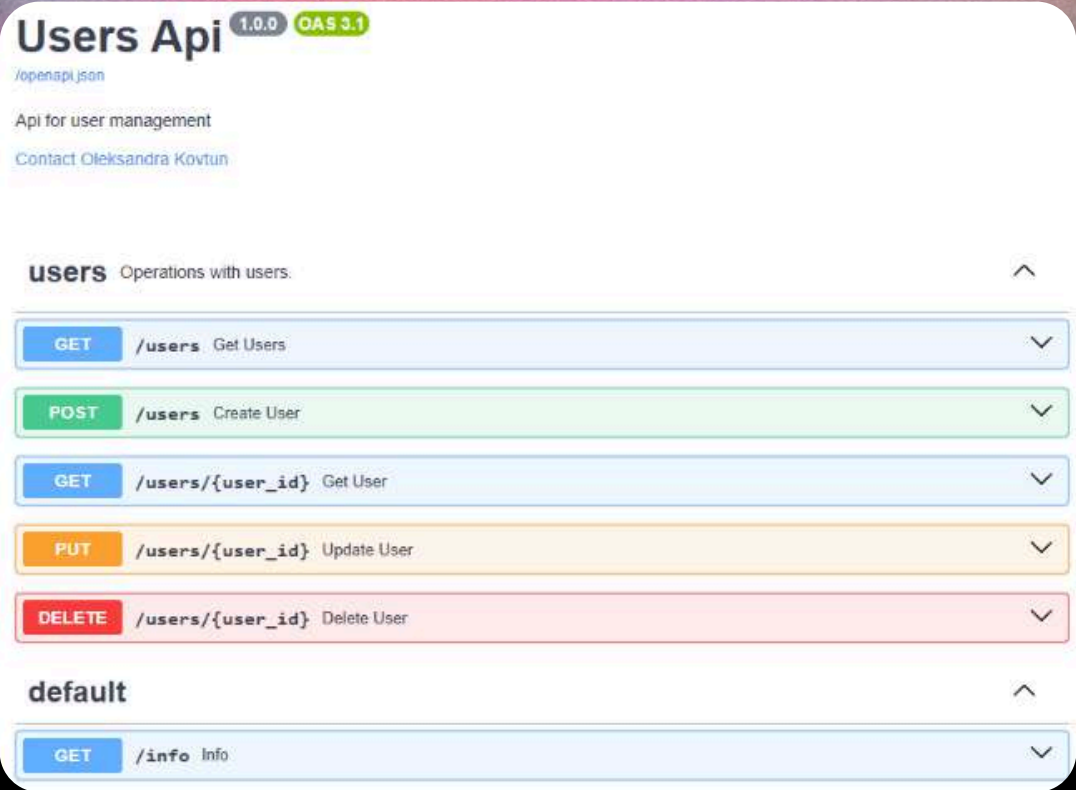
Валідація на рівні бази даних:

Для підтвердження цілісності системи ми звірили результати API з ізольованою базою даних PostgreSQL. У DBeaver чітко видно таблицю games, де зберігаються відповідні записи.

04

Успішне завершення циклу:

Дані, отримані через клієнтський запит, на 100% збігаються з фізичним станом бази даних.



Демонстрація розгорнутого функціоналу

01

Перевірка через Swagger UI:

Після успішного деплою ми виконали тестування доданої фічі безпосередньо на живому середовищі. Запит до нового ендпоінту GET /api/chats успішно направлено до кластера.

02

Коректна відповідь API:

Сервіс Chats API опрацював запит, повернув статус 200 OK та очікуваний масив даних у форматі JSON (інформацію про ігри користувача), що повністю підтверджує працездатність оновленого коду.

03

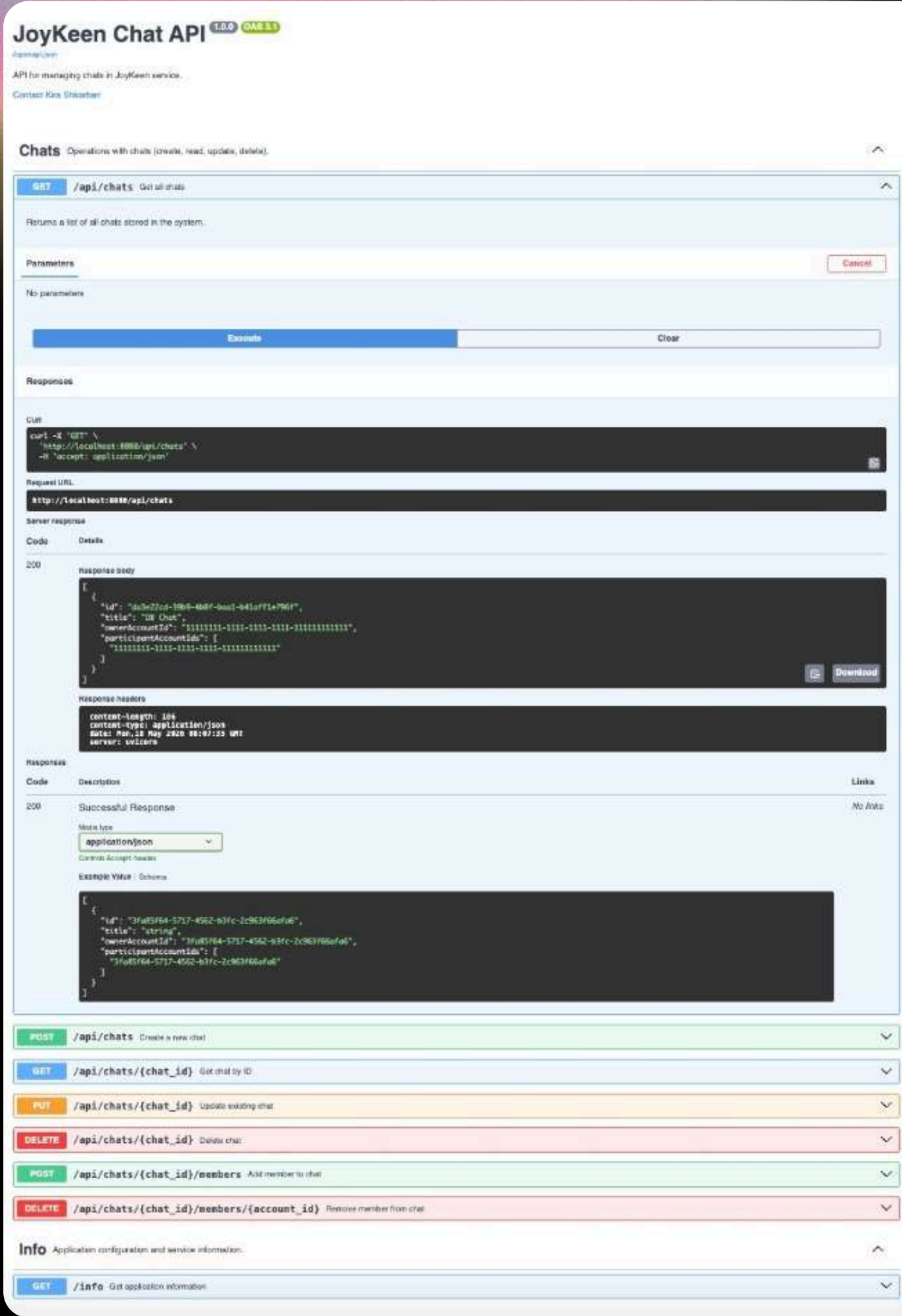
Валідація на рівні бази даних:

Для підтвердження цілісності системи ми звірили результати API з ізольованою базою даних PostgreSQL. У DBeaver чітко видно таблицю games, де зберігаються відповідні записи.

04

Успішне завершення циклу:

Дані, отримані через клієнтський запит, на 100% збігаються з фізичним станом бази даних.



Архітектура баз даних

	game_id	AZ title	AZ description	min_players	created_at
1	cb1bd5c8-15e2-475f-8961-2aa66992ab1c	The Walking Dead Game	Постапокаліптична пригодницька гра	2	2026-05-05 19:11:38.086 +0300
2	d34479c9-e9de-47e4-9c4f-80a475f3fbcf	Kubs	string	2	2026-05-06 14:45:38.294 +0300
3	c3721005-4171-48f9-82c5-95eec0dd6388	12121	sdaieaas	3	2026-05-12 09:24:12.889 +0300
4	2f27f160-6f63-46a0-e48e-7ac79b7162e8	Chess	Chess with 2 players	2	2026-05-18 11:10:53.262 +0300

	user_id	AZ firstName	AZ lastName	AZ nickname	AZ email	AZ password	isActive	created_at	updated_at
1	bccdb077-2ed9-48ee	Олена	Ткаченко	lena_tk	lena@example.co	pass123	[v]	026-05-06 07:36:03.452	2026-05-06 07:36:03.452
2	46f63892-71f0-4e8e-9	Максим	Григоренко	max_g	max@example.co	pass123	[v]	026-05-06 07:36:03.452	2026-05-06 07:36:03.452
3	b60acf90-205c-44e3-	Вікторія	Мельник	vika_m	vika@example.co	pass123	[]	026-05-06 07:36:03.452	2026-05-06 07:36:03.452
4	9440dc60-3bcc-4b45-	Андрій	Бойко	andrew_b	andrew@example	pass123	[v]	026-05-06 07:36:03.452	2026-05-06 07:36:03.452
5	79cabcc8-bcf4-428c-	Катерина	Лисенко	katya_l	katya@example.c	pass123	[v]	026-05-06 07:36:03.452	2026-05-06 07:36:03.452

chats × *chat_db> Script

PropertiesDataDiagram

Show SQL Enter a SQL expression to filter results (use Ctrl+Space)

	AZ id	AZ title	AZ owner_account_id
1	c3bc3521-63ce-4386-8c	Demo Chat	11111111-1111-1111-1111-11'
2	3608a95f-57ad-4689-b2	API Chat	11111111-1111-1111-1111-11'
3	4d50088c-c6c1-4e6f-93	New Chat	11111111-1111-1111-1111-11'
4	b345f981-b652-4204-bc	Cool Chat	11111111-1111-1111-1111-11'

Ізоляція стану (State Isolation):

01 Замість використання єдиної монолітної бази даних, ми впровадили патерн "Database-per-service". Як видно на демонстрації (DBeaver), кожен мікросервіс проєкту JoyKeen (Account, User, Game, Chat) керує власною ізольованою схемою або базою даних.

Слабка зв'язність (Loose Coupling):

02 Сервіси не мають прямого доступу до баз даних один одного. Будь-яка взаємодія, наприклад, створення ігрової сесії (games) для певного користувача (users), відбувається виключно через REST API виклики між самими сервісами. Це унеможливорює приховані залежності на рівні SQL-запитів.

Стійкість та масштабування:

03 Такий підхід гарантує високу відмовостійкість системи. Якщо база даних chat-арі вийде з ладу або перевантажиться, функціонал user-арі та game-арі продовжить працювати у штатному режимі. Крім того, це дозволяє гнучко масштабувати сховища для кожного сервісу окремо залежно від навантаження.

Міграції (GitOps ready):

04 Структура кожної бази (таблиці, індекси) підтримується в актуальному стані завдяки автоматизованим Job-ам з використанням Alembic, що виконуються безпосередньо в кластері Kubernetes перед стартом застосунків.

Pipelines

Recent All Runs

Recently run pipelines

Pipeline	Last run
✓ chat_api	#20260511.6 • Upda Individual CI for
✓ game_api	#20260505.11 • 12 л Individual CI for
✓ users_api	#20260504.11 • lab Individual CI for
✓ account_api	#20260429.15 • PR1 Individual CI for

CI Pipeline: Автоматизація в Azure DevOps

Page 08

01

Комплексна автоматизація (Azure DevOps):

Процес Continuous Integration повністю автоматизовано за допомогою Azure Pipelines. При злитті коду в гілку main запускається пайплайн, що складається з ізольованих етапів: Pre-Build (аналіз та розрахунок версії), Project Build and Test (прогін юніт-тестів), Docker Build and Push, та пакування Helm-чарту. На демонстрації видно успішне виконання всіх стадій (100% passed).

02

Управління Docker-артефактами:

Після успішного проходження тестів система збирає Docker-образ сервісу та автоматично присвоює йому розрахований тег версії (наприклад, 1.0.1 та latest). Готовий образ завантажується до зовнішнього Container Registry (Docker Hub), що гарантує збереження незмінного артефакту для подальшого розгортання в будь-якому середовищі.

03

Пакування Helm-чартів (OCI Artifacts):

Окрім Docker-образів, пайплайн автоматизує роботу з інфраструктурним кодом. Спеціальний Bash-скрипт вичитує версію з файлу Chart.yaml, пакує Helm-чарт у формат .tgz та пушить його у Registry як OCI-артефакт. Це гарантує, що кожна версія коду має відповідну їй версію маніфестів розгортання.

04

GitOps Bridge (Trigger Auto-deployment):

Найважливіший крок пайплайну — автоматичний зв'язок між CI та CD. Після створення артефактів спеціальна джоба робить checkout нашого state-репозиторію (argocd_state), динамічно оновлює поле targetRevision на нову версію чарту та автоматично робить commit і push. Це миттєво сигналізує Argo CD про необхідність розпочати деплой.

CI Pipeline: Автоматизація в Azure DevOps (account_api)

Individual CI by  Oleksandr Linkov

Repositories 3

account_api , +2

See Sources card for details

Time started and elapsed

29 apr. at 22:27

3m 17s

Related

0 work items

3 published; 2 consumed

View 4 changes

Parameters

Tests and coverage

100% passed

0% covered

Warnings 4

Support for class-based `config` is deprecated, use ConfigDict instead. Deprecatcd in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at <https://errors.pydantic.dev/2.12/migration/>

Project: Build and Test • Build And Test • Run Unit Tests

Support for class-based `config` is deprecated, use ConfigDict instead. Deprecatcd in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at <https://errors.pydantic.dev/2.12/migration/>

Project: Build and Test • Build And Test • Run Unit Tests

No data was written into the file /home/vsts/work/_temp/task_outputs/build_1777490980612.txt

Docker: Build and Push • Build and Push • Build Docker Image

No data was written into the file /home/vsts/work/_temp/task_outputs/build_1777490929188.txt

Docker: Build and Push • Build and Push • Build Docker Image

Stages Jobs

Pre-Build

1 job completed

8s

Project: Build and Test

1 job completed

23s

100% tests passed

Docker: Build and Push

1 job completed

50s

1 artifact

Helm_Chart

1 job completed

17s

1 artifact

Post-Build

1 job completed

5s

Docker: Build and Push

1 job completed

45s

1 artifact

Завдання №3.1: Bash-скрипт перевірки версії

Starting: Завдання %3.1: Bash-скрипт перевірки версії

=====

Task : Bash

Description : Run a Bash script on macOS, Linux, or Windows

Version : 3.268.1

Author : Microsoft Corporation

Help : <https://docs.microsoft.com/azure/devops/pipelines/tasks/utility/bash>

=====

Generating script.

===== Starting Command Output =====

/usr/bin/bash /home/vsts/work/_temp/51b57cce-4bac-411e-ab83-69e8db069b67.sh

Знайдена версія чарта: 16.0.145

Finishing: Завдання %3.1: Bash-скрипт перевірки версії

Завдання №3.b: Push Helm Chart to Docker Hub

Starting: Завдання %3.b: Push Helm Chart to Docker Hub

=====

Task : Bash

Description : Run a Bash script on macOS, Linux, or Windows

Version : 3.268.1

Author : Microsoft Corporation

Help : <https://docs.microsoft.com/azure/devops/pipelines/tasks/utility/bash>

=====

Generating script.

===== Starting Command Output =====

/usr/bin/bash /home/vsts/work/_temp/04fe4ceb-d06d-444d-ad00-6f1a2e98ce56.sh

Login Succeeded

Вміст папки артефактів:

total 4.0K

-rw-r--r-- 1 vsts vsts 2.6K Apr 29 19:30 joykeen-chart-16.0.145.tgz

Pushed: registry-1.docker.io/sasha333/joykeen-chart:16.0.145

Digest: sha256:7c4df7fd60e10ad495025173c8f49484f132a38ef1e79ff7600726b1e0db8a3f

Finishing: Завдання %3.b: Push Helm Chart to Docker Hub

CI Pipeline: Автоматизація в Azure DevOps (game_api)

Page 10

Individual CI by  Олександр Мірошніченко

Repositories 2

game_api , +1

See Sources card for details

Time started and elapsed

6 мая at 01:52

4m 39s

Related

0 work items

2 published; 1 consumed

Tests and coverage

100% passed

0% covered

View change

Warnings 2

No data was written into the file /home/vsts/work/_temp/task_outputs/build_1778021623064.txt

Docker: Build and Push • Build and Push • Build Docker Image

No data was written into the file /home/vsts/work/_temp/task_outputs/build_1778021747055.txt

Docker: Build and Push • Build and Push • Build Docker Image

Stages Jobs

Pre-Build

1 job completed

9s

Project: Build and Test

1 job completed

28s

100% tests passed

Docker: Build and Push

1 job completed

1m 9s

1 artifact

Docker: Build and Push

1 job completed

1m 10s

1 artifact

Helm Chart: Pack

1 job completed

5s

Helm Chart Build & P...

1 job completed

13s

Зчитати версію з Chart.yaml

1 Starting: Зчитати версію з Chart.yaml

2 =====

3 Task : Bash

4 Description : Run a Bash script on macOS, Linux, or Windows

5 Version : 3.268.1

6 Author : Microsoft Corporation

7 Help : <https://docs.microsoft.com/azure/devops/pipelines/tasks/utility/bash>

8 =====

9 Generating script.

10 ===== Starting Command Output =====

11 /usr/bin/bash /home/vsts/work/_temp/d97c8bd5-c964-4748-a7fe-af9429899b08.sh

12 Виявлена версія чарта: 16.0.0

13

14 Finishing: Зчитати версію з Chart.yaml

Завантажити Chart у Docker Hub

1 Starting: Завантажити Chart у Docker Hub

2 =====

3 Task : Bash

4 Description : Run a Bash script on macOS, Linux, or Windows

5 Version : 3.268.1

6 Author : Microsoft Corporation

7 Help : <https://docs.microsoft.com/azure/devops/pipelines/tasks/utility/bash>

8 =====

9 Generating script.

10 ===== Starting Command Output =====

11 /usr/bin/bash /home/vsts/work/_temp/1c75d0b8-23d8-497d-9977-be83def2d63b.sh

12 Login Succeeded

13 Pushed: registry-1.docker.io/fltt/joykeen-chart:16.0.0

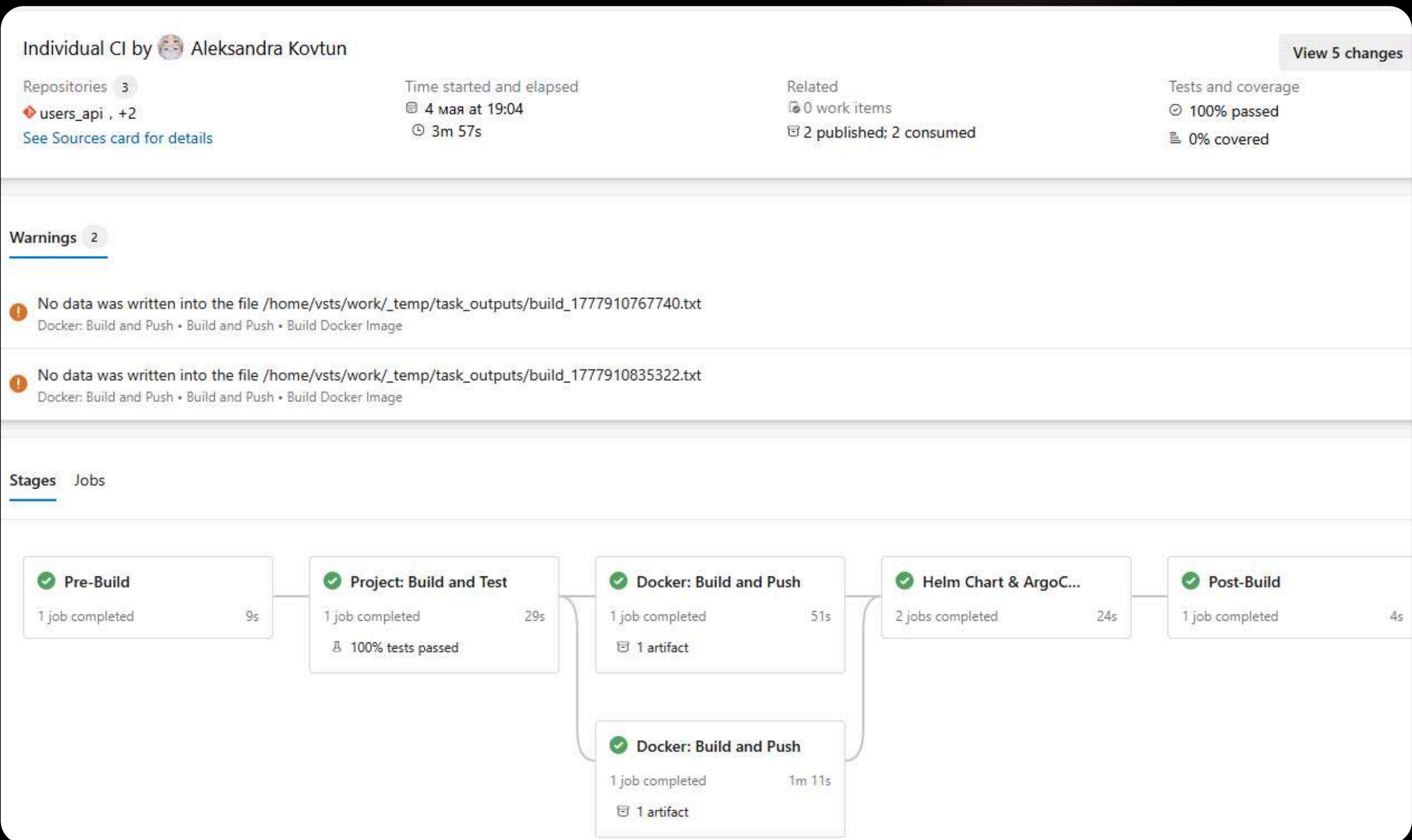
14 Digest: sha256:01d92305ff5afbc788c5babd5f124e911cdb5a090814c5cdedc630197710d13d

15

16 Finishing: Завантажити Chart у Docker Hub

CI Pipeline: Автоматизація в Azure DevOps (users_api)

Page 11

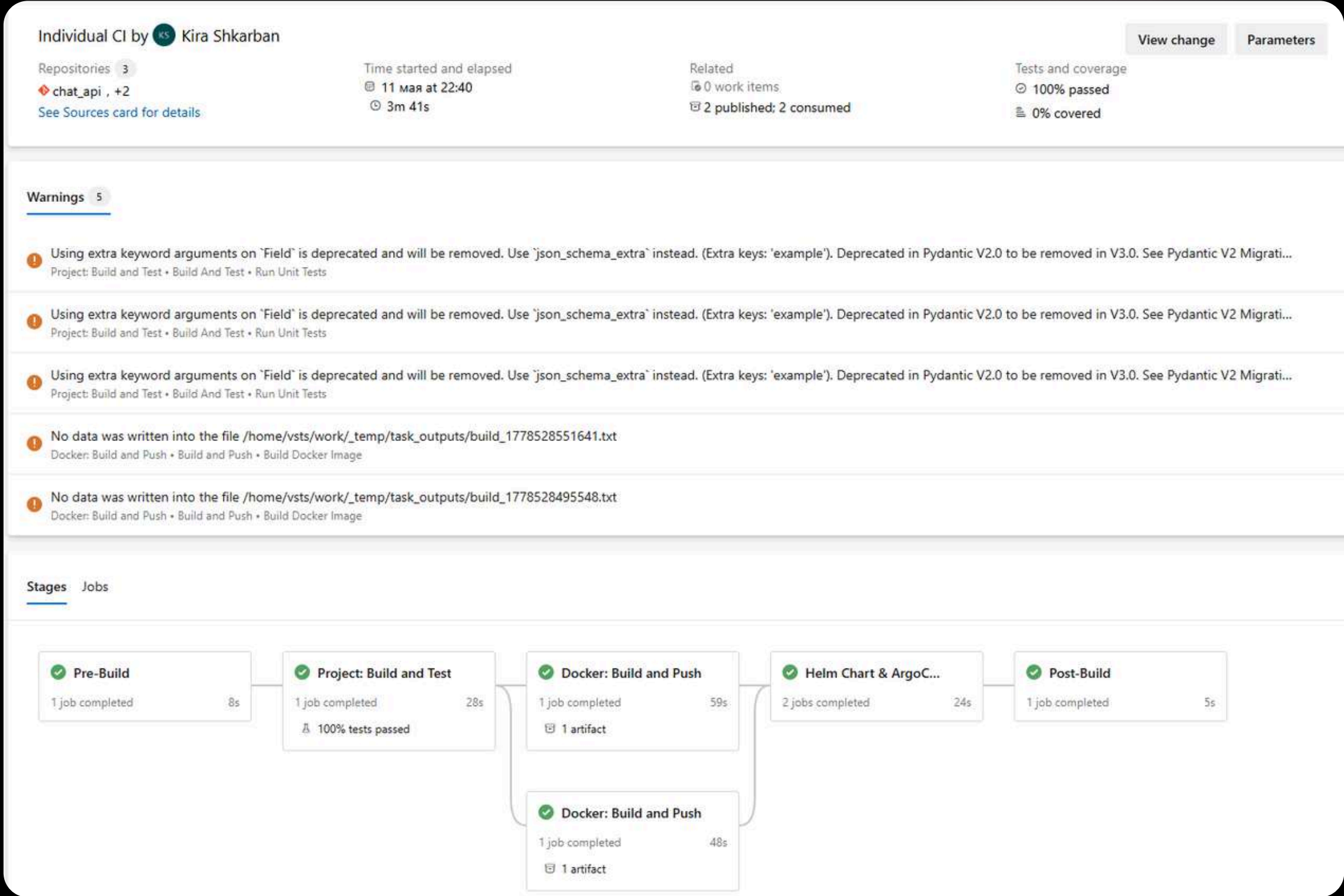


Bash: Task 3 Final Fix

```
1 Starting: Bash: Task 3 Final Fix
2 =====
3 Task      : Bash
4 Description : Run a Bash script on macOS, Linux, or Windows
5 Version    : 3.268.1
6 Author     : Microsoft Corporation
7 Help       : https://docs.microsoft.com/azure/devops/pipelines/tasks/utility/bash
8 =====
9 Generating script.
10 ===== Starting Command Output =====
11 /usr/bin/bash /home/vsts/work/_temp/61249461-bdbc-44ec-863e-a8d2e76fe2f0.sh
12 Знайдено чарт: users-api-chart, Версія: 16.0.6
13 Login Succeeded
14 Checking if oci://registry-1.docker.io/hiimpala1967/users-api-chart:16.0.6 exists...
15 Successfully packaged chart and saved it to: /home/vsts/work/1/s/users-api-chart-16.0.6.tgz
16 Pushing archive to oci://registry-1.docker.io/hiimpala1967
17 Pushed: registry-1.docker.io/hiimpala1967/users-api-chart:16.0.6
18 Digest: sha256:c7933c35256cb0945e758ea744a236cae7f3cb67552bfcf3846b4e2f3d391005
19
20 Finishing: Bash: Task 3 Final Fix
```

CI Pipeline: Автоматизація в Azure DevOps (chat_api)

Page 12



✓ Read Chart version

```
1 Starting: Read Chart version
2 =====
3 Task      : Command line
4 Description : Run a command line script using Bash on Linux and macOS and cmd.exe on Windows
5 Version    : 2.268.0
6 Author     : Microsoft Corporation
7 Help       : https://docs.microsoft.com/azure/devops/pipelines/tasks/utility/command-line
8 =====
9 Generating script.
10 ===== Starting Command Output =====
11 /usr/bin/bash --noprofile --norc /home/vsts/work/_temp/a4b50de6-2b00-4f64-bc71-8cb438c90560.sh
12 Chart version: 16.0.4
13
14 Finishing: Read Chart version
```

✓ Build and Push Helm Chart

```
1 Starting: Build and Push Helm Chart
2 =====
3 Task      : Command line
4 Description : Run a command line script using Bash on Linux and macOS and cmd.exe on Windows
5 Version    : 2.268.0
6 Author     : Microsoft Corporation
7 Help       : https://docs.microsoft.com/azure/devops/pipelines/tasks/utility/command-line
8 =====
9 Generating script.
10 ===== Starting Command Output =====
11 /usr/bin/bash --noprofile --norc /home/vsts/work/_temp/a18f7e43-e69a-4b05-be9f-adc87f8de7a1.sh
12 Version 16.0.4 is new. Proceeding with build and push...
13 Successfully packaged chart and saved it to: /tmp/chat-api-16.0.4.tgz
14 Pushed: registry-1.docker.io/kiraashkarban/chat-api:16.0.4
15 Digest: sha256:4b97f7fc64c68582177ab98ff3ba93c8f41edbb6759d58cdb0c2867b9050a6e4
16
17 Finishing: Build and Push Helm Chart
```

CD Pipeline: GitOps через Argo CD

Let's get stuff deployed!



01

Принцип Single Source of Truth:

Ми впровадили GitOps-підхід, де Git-репозиторій є єдиним джерелом істини для інфраструктури. Argo CD безперервно відстежує стан нашого state-репозиторію та автоматично виявляє розбіжності між задекларованим кодом та реальним станом кластера Kubernetes.

02

Автоматична синхронізація (Sync):

На скріншотах зафіксовано статус Synced. Це означає, що як тільки CI-пайплайн оновив версію Helm-чарту (наприклад, до 16.0.x), Argo CD миттєво підхопив ці зміни та розгорнув нову версію мікросервісів у кластері без жодного ручного втручання.

03

Візуалізація дерева ресурсів:

Завдяки інтерфейсу Argo CD ми бачимо повну ієрархію ресурсів для кожного сервісу: від об'єктів Application та Service до конкретних Deployment, ReplicaSet та Pods. Також візуалізовано супутні ресурси: ConfigMaps, Secrets та PersistentVolumeClaims.

04

Контроль стану (Health Status):

Статус Healthy (зелене серце) на всіх компонентах системи підтверджує, що нова версія не просто розгорнута, а й успішно пройшла всі перевірки працездатності (Liveness/Readiness probes) і готова до обробки трафіку в реальному часі.

CD Pipeline: GitOps через Argo CD

(account_api)

Page 14

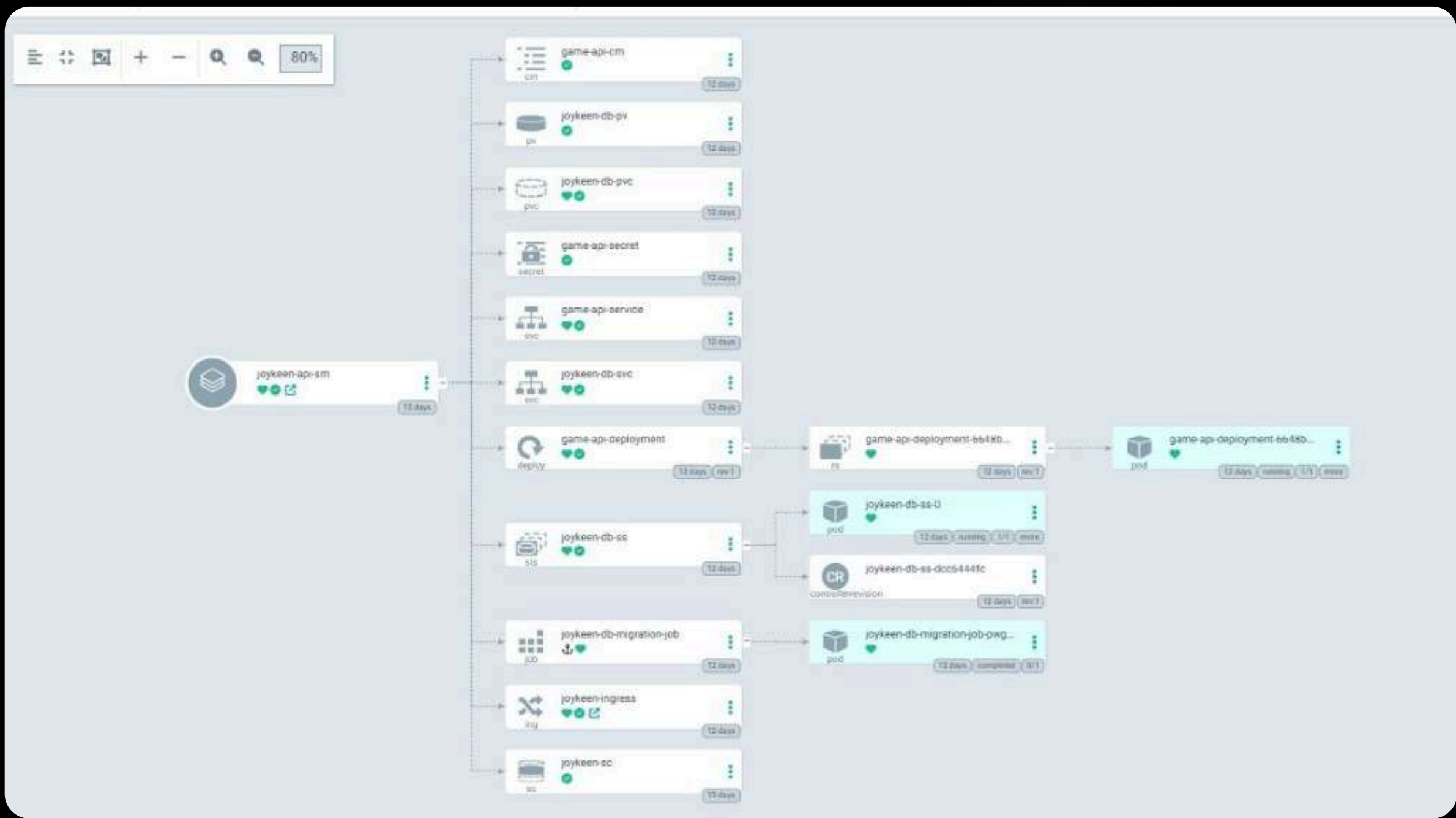
The screenshot displays the Argo CD v3.3.8 interface for the 'joykeen-api' application. The left sidebar contains navigation links for Applications, Settings, User Info, and Documentation, along with resource filters for NAME, KINDS, SYNC STATUS, and HEALTH STATUS. The main panel shows the application's status as 'Healthy' and 'Synced' to revision 16.0.145. A 'LAST SYNC' section indicates a successful sync at 20:43:55 GMT+0200 on Wed Apr 29 2026. The application's manifest is visualized as a dependency graph, showing resources like 'account-secrets', 'account-api-svc', 'account-db-svc', 'account-api-deployment', 'account-db', 'joykeen-api-migration', and 'joykeen-ingress' and their relationships.

This screenshot shows the Argo CD v3.3.8 interface with two application cards displayed. The 'joykeen-api' card shows it is 'Healthy' and 'Synced' to revision 16.0.0, with a target revision of 16.0.0. The 'joykeen-manager' card shows it is 'Healthy' and 'Synced' to revision HEAD, with a target revision of HEAD. Both cards include buttons for SYNC, REFRESH, and DELETE. The left sidebar shows the same navigation and filter options as the previous screenshot.

CD Pipeline: GitOps через Argo CD

(game_api)

Page 15



The image shows the Argo CD Application Details page for the application 'joykeen-api-sm'. The page displays the application's configuration, including:

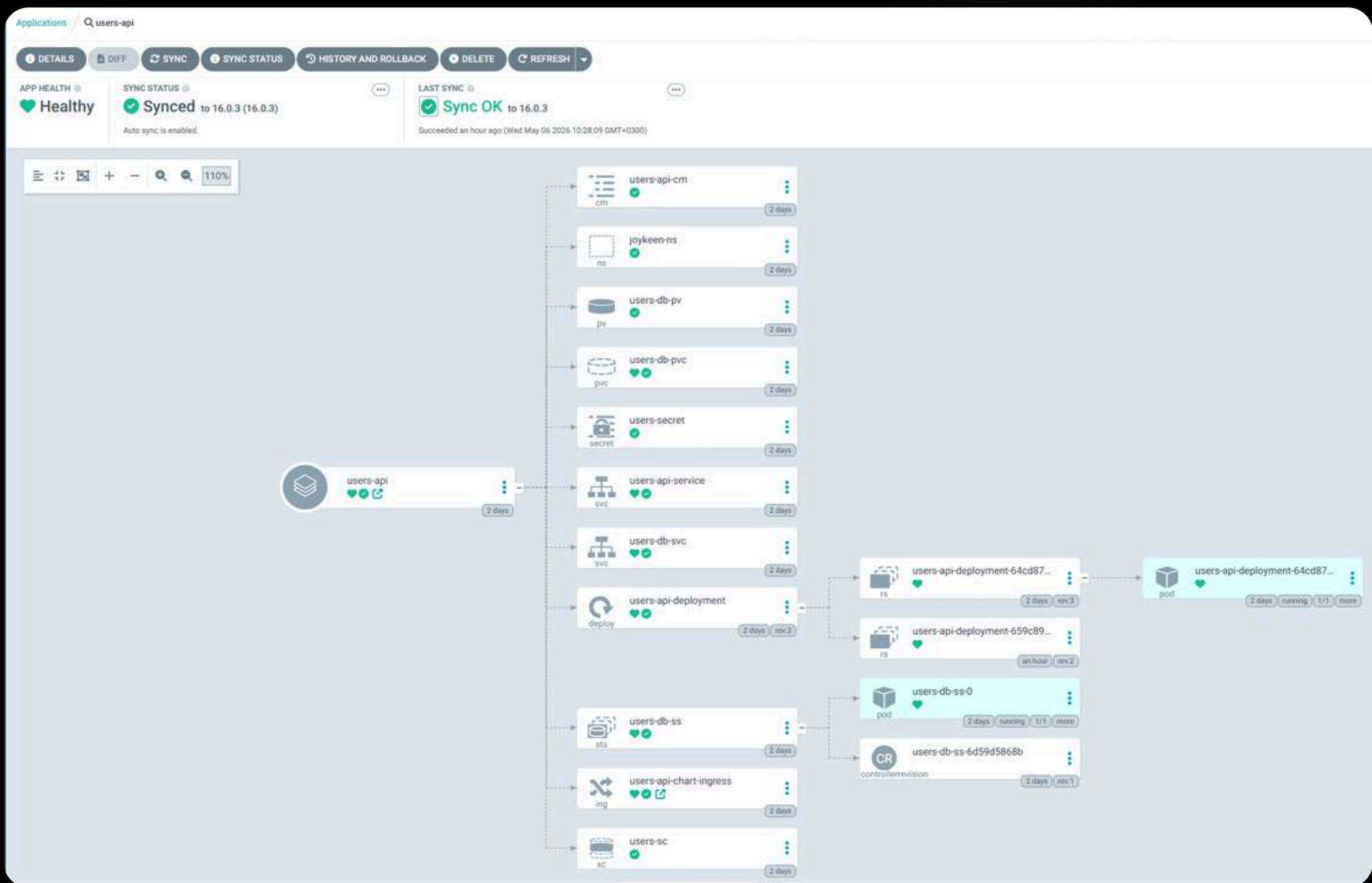
- Project: default
- Labels: (empty)
- Status: Healthy Synced
- Repository: registry-1.docker.io/f1tt
- Target R...: 16.0.0
- Chart: joykeen-chart
- Destinat...: in-cluster
- Namesp...: game-api-ns
- Created ...: 05/06/2026 00:25:15 (12 days ago)
- Last Sync: 05/12/2026 08:47:19 (6 days ago)

The page also includes a sidebar with navigation options: Applications, Settings, User Info, and Documentation. The main content area shows the application's status, including Sync Status (Unknown, Synced, OutOfSync) and Health Status (Progressing, Suspended, Healthy, Degraded, Missing, Unknown).

CD Pipeline: GitOps через Argo CD

(users_api)

Page 16



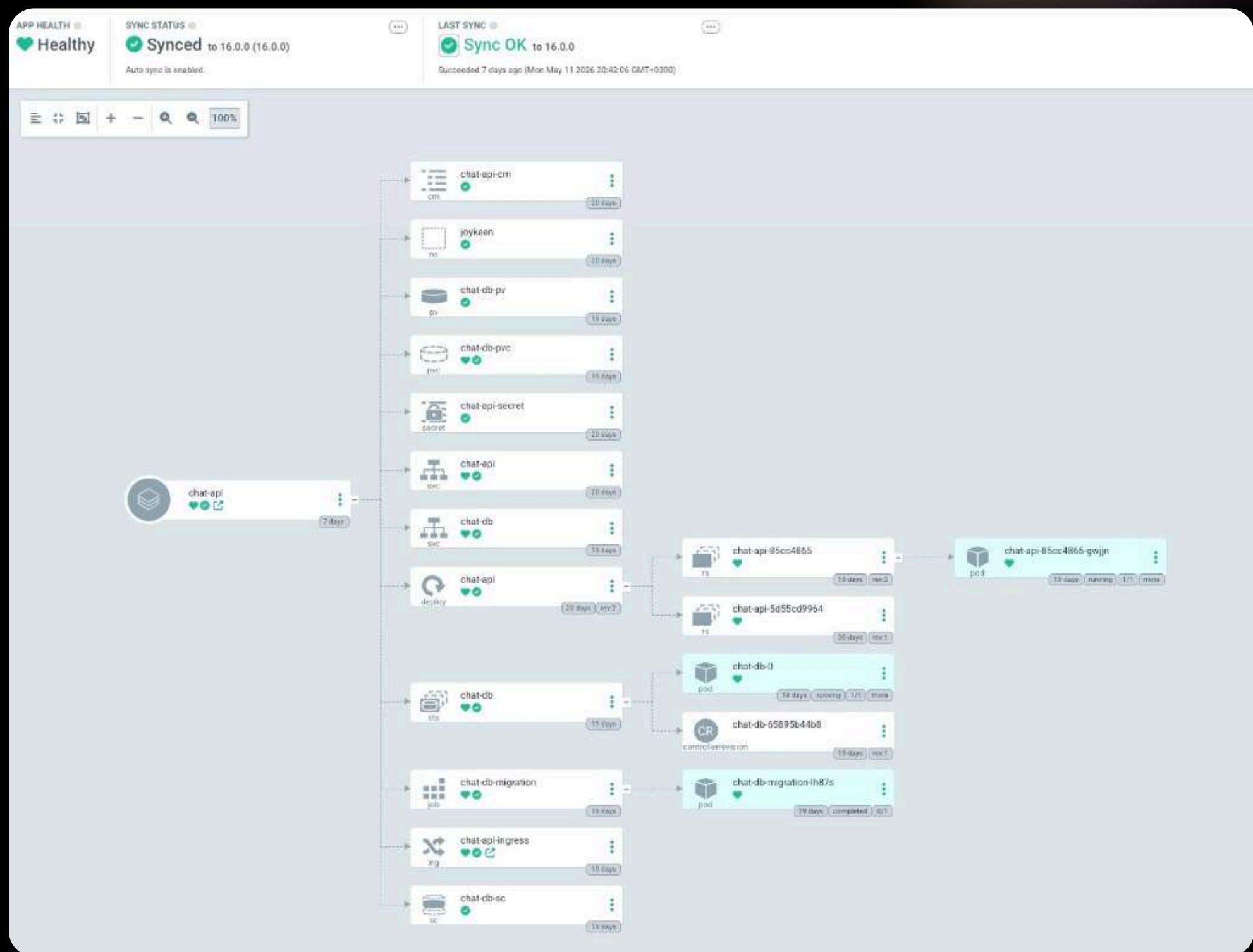
The screenshot shows the Argo CD web interface for the 'users-api' application. The top navigation bar includes buttons for NEW APP, SYNC APPS, and REFRESH APPS. The application's health is 'Healthy' and its sync status is 'Synced'. The main area displays the application's details, including its project, repository, target revision, chart, destination, namespace, creation time, and last sync time.

Application filters	Count
★ Favorites Only	0
SYNC STATUS	
Unknown	0
Synced	1
OutOfSync	0
HEALTH STATUS	
Progressing	0
Suspended	0
Healthy	1

CD Pipeline: GitOps через Argo CD

(chat_api)

Page 17



Applications

Search applications...

chat-api

Project: default

Labels:

Status: Healthy Synced

Repository: registry-1.docker.io

Target Re... 16.0.0

Chart: kiraashkarban/chat-api

Destinati... in-cluster

Namespa... joykeen

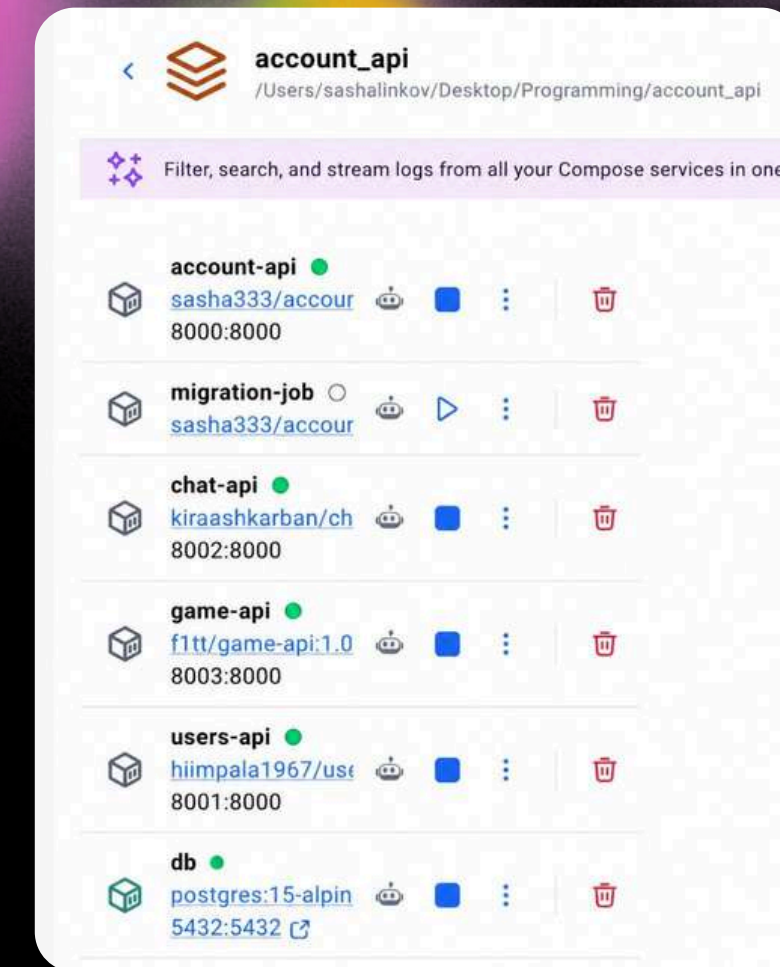
Created At: 05/11/2026 20:42:00 (2 hours ago)

Last Sync: 05/11/2026 20:42:06 (2 hours ago)

SYNC Refresh Close

Моніторинг ресурсів у кластері Kubernetes

Page 18



01

Статус контейнерів (Pods State):

Після того як Argo CD застосував оновлені маніфести, Kubernetes автоматично провів rolling update для відповідних Deployment-ів. Старі версії контейнерів були плавно зупинені, а нові поди мікросервісів перейшли у статус Running. На скріншоті з OpenLens видно стабільну роботу всіх запущених реплік.

02

Ізоляція у просторі імен (Namespace):

Усі компоненти платформи розгорнуті у виділеному Namespace кластера. Це забезпечує надійну ізоляцію інфраструктури проєкту, безпеку внутрішньої мережевої взаємодії між сервісами (Account, Users, Chat, Game) та дозволяє гнучко керувати доступами й квотами ресурсів.

03

Контроль життєвого циклу (Lifecycle):

Метрика Age (час роботи подів) у таблиці OpenLens чітко підтверджує факт нещодавнього оновлення системи в результаті нашого деплою. Нові контейнери успішно пройшли перевірки працездатності (Readiness та Liveness probes) і миттєво включилися в обробку трафіку.

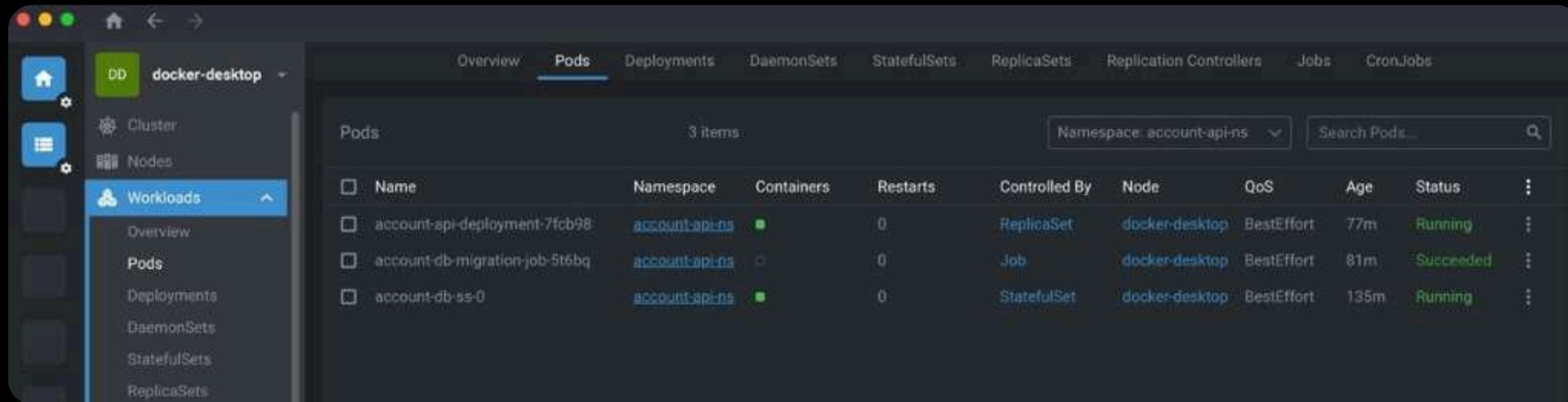
04

Спостережуваність системи (Observability):

Використання OpenLens як інструменту моніторингу дає команди повний контроль над кластером у реальному часі. Ми можемо відстежувати утилізацію ресурсів (CPU/Memory), переглядати логи контейнерів, а також контролювати стан супутніх об'єктів, таких як StatefulSets для баз даних PostgreSQL.

Моніторинг ресурсів у кластері Kubernetes Page 19

(account_api)

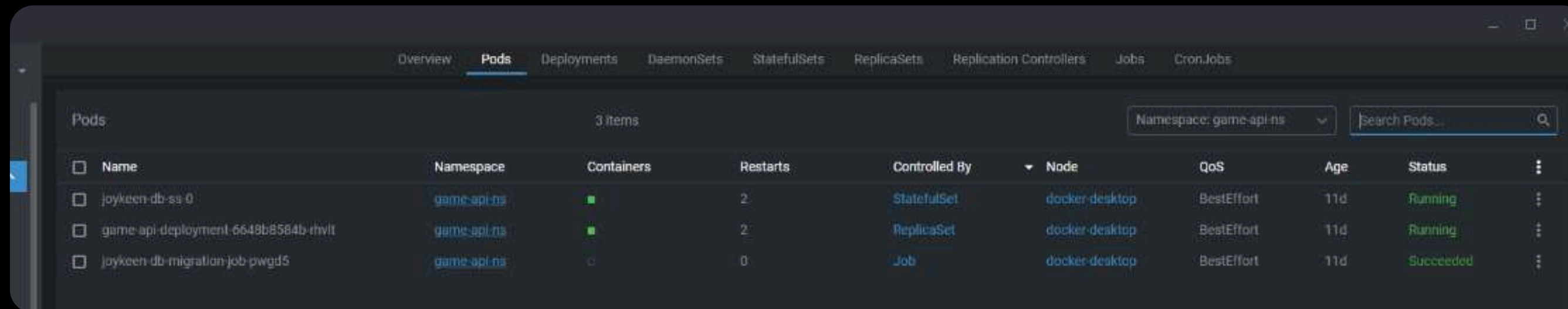


The screenshot shows the Kubernetes Dashboard interface. The left sidebar has a 'Workloads' menu item selected. The main panel is titled 'Pods' and shows a list of 3 items in the 'account-api-ns' namespace. The table columns are: Name, Namespace, Containers, Restarts, Controlled By, Node, QoS, Age, and Status. The first pod is 'account-api-deployment-7fcb98' with 0 restarts, controlled by a 'ReplicaSet', and is in a 'Running' state. The second pod is 'account-db-migration-job-5t6bq' with 0 restarts, controlled by a 'Job', and is in a 'Succeeded' state. The third pod is 'account-db-ss-0' with 0 restarts, controlled by a 'StatefulSet', and is in a 'Running' state.

Name	Namespace	Containers	Restarts	Controlled By	Node	QoS	Age	Status
account-api-deployment-7fcb98	account-api-ns	1	0	ReplicaSet	docker-desktop	BestEffort	77m	Running
account-db-migration-job-5t6bq	account-api-ns	1	0	Job	docker-desktop	BestEffort	81m	Succeeded
account-db-ss-0	account-api-ns	1	0	StatefulSet	docker-desktop	BestEffort	135m	Running

Моніторинг ресурсів у кластері Kubernetes Page 20

(game_api)



The screenshot shows the Kubernetes dashboard interface. At the top, there are tabs for Overview, Pods, Deployments, DaemonSets, StatefulSets, ReplicaSets, Replication Controllers, Jobs, and CronJobs. The 'Pods' tab is selected. Below the tabs, there is a header for 'Pods' with '3 items' and a search bar. The main content is a table listing the pods in the 'game-api' namespace.

☐	Name	Namespace	Containers	Restarts	Controlled By	Node	QoS	Age	Status	
☐	joykeen-db-ss-0	game-api-ns		2	StatefulSet	docker-desktop	BestEffort	11d	Running	
☐	game-api-deployment-6649b8584b-rhvt	game-api-ns		2	ReplicaSet	docker-desktop	BestEffort	11d	Running	
☐	joykeen-db-migration-job-pwgd5	game-api-ns		0	Job	docker-desktop	BestEffort	11d	Succeeded	

Моніторинг ресурсів у кластері Kubernetes Page 21

(users_api)

Pods

4 items

Namespace: joykeen-ns

Search Pods...

☐	Name	Namespace	Containers	Restarts	Controlled By	Node	QoS	Age	Status	
☐	users-api-deployment-59d6b4f8f5-8	joykeen-ns		0	ReplicaSet	docker-desktop	BestEffort	75m	Running	
☐	users-db-migration-job-d8fkh	joykeen-ns		0	Job	docker-desktop	BestEffort	54m	Succeeded	
☐	users-db-ss-0	joykeen-ns		0	StatefulSet	docker-desktop	BestEffort	75m	Running	

Моніторинг ресурсів у кластері Kubernetes Page 22

(chat_api)

Overview **Pods** Deployments DaemonSets StatefulSets ReplicaSets Replication Controllers Jobs CronJobs

Pods 3 items Namespace: joykeen Search Pods...

☐	Name	Namespace	Containers	Restarts	Controlled By	Node	QoS	Age	Status	⋮
☐	chat-api-85cc4865-gwjn	joykeen		0	ReplicaSet	docker-desktop	BestEffort	19d	Running	⋮
☐	chat-db-0	joykeen		0	StatefulSet	docker-desktop	BestEffort	19d	Running	⋮
☐	chat-db-migration-lh87s	joykeen		0	Job	docker-desktop	BestEffort	19d	Succeeded	⋮

Підсумки

01

Надійна автоматизація CI:

Завдяки Azure Pipelines ми налаштували безперебійний процес інтеграції. Система автоматично розраховує версії, проганяє тести та пакує готові артефакти (Docker-образи та OCI Helm-чарти) при кожному оновленні коду.

02

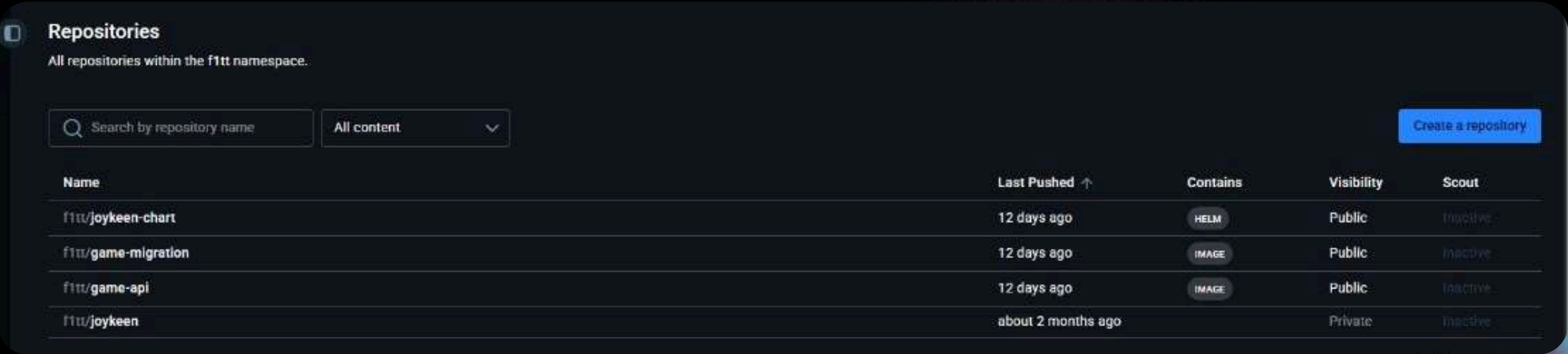
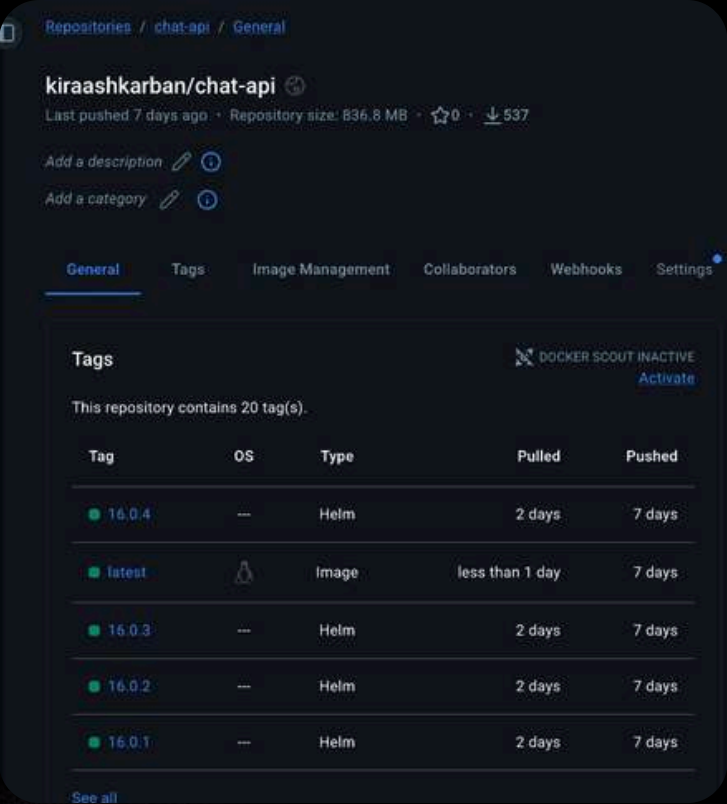
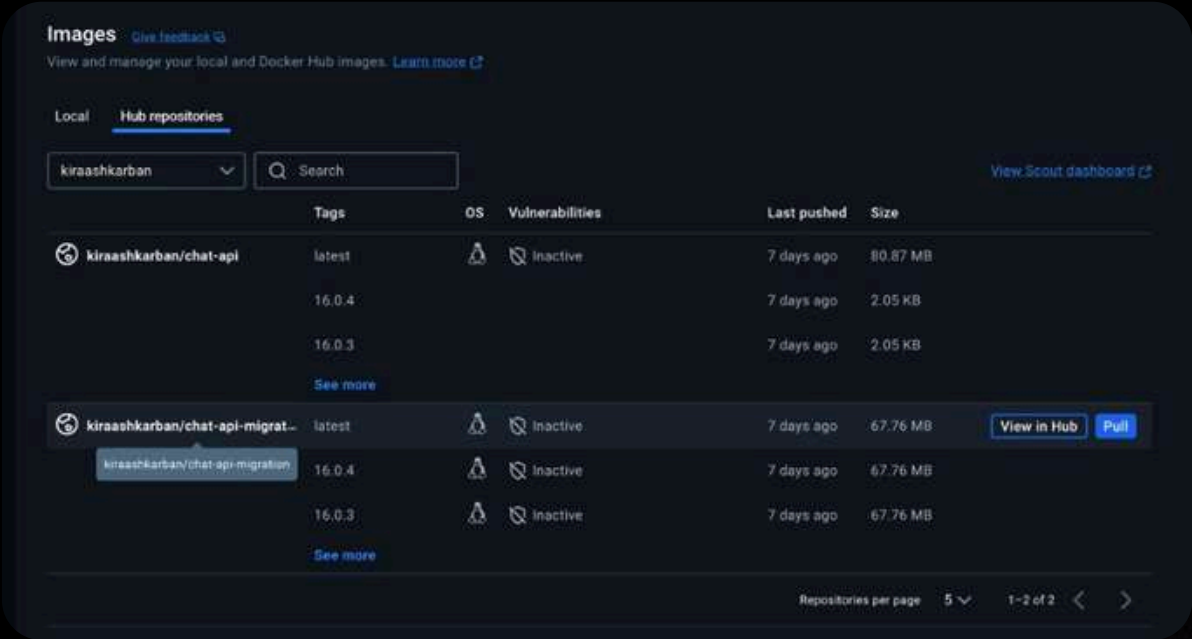
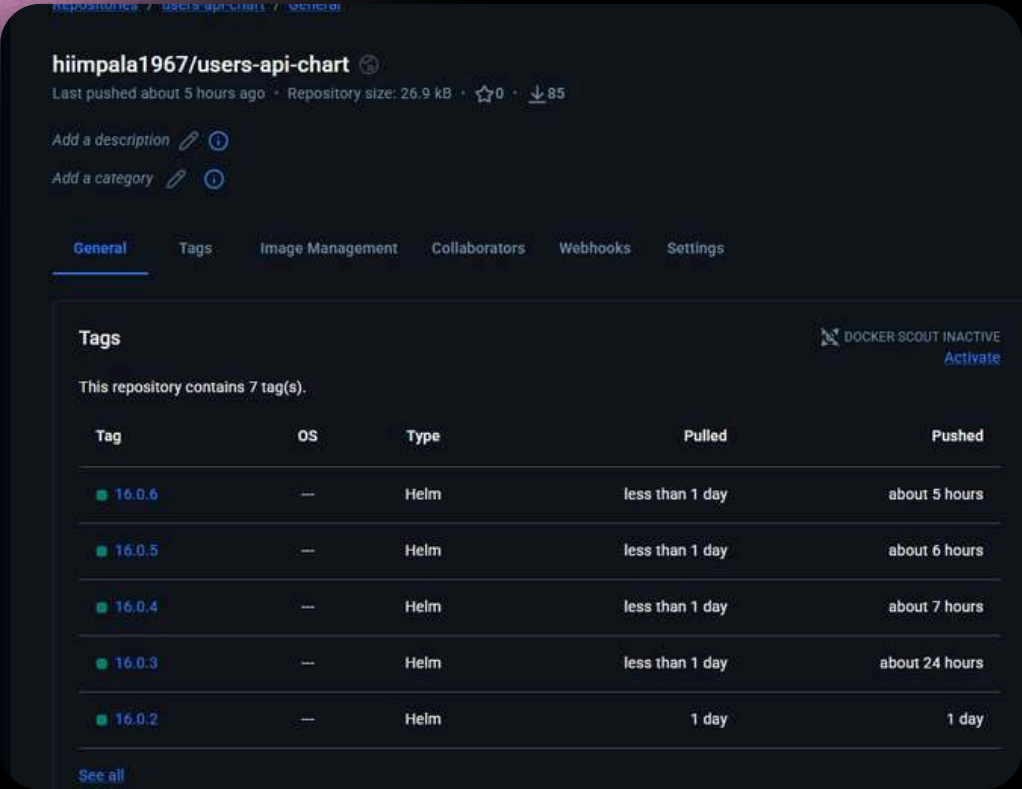
GitOps-орієнтований CD:

Впровадження Argo CD дозволило повністю автоматизувати доставку маніфестів у кластер. Ми виключили ручне втручання: розгортання відбувається виключно через синхронізацію зі state-репозиторієм.

03

Стабільна робота в Kubernetes:

Мікросервісна архітектура JoyKeep успішно функціонує в K8s. Всі сервіси та їхні ізольовані бази даних працюють у штатному режимі, що підтверджено безперервним моніторингом через OpenLens.



Дякуємо за увагу!